

Developing a domain-specific LLM through self-instruct fine-tuning

Application to reliability engineering

Alex Dalban Elora Drouilhet

CentraleSupélec

Supervisors: Dr. Zhiguo Zeng & PhD Candidate Jean Meunier-Pion

December 14, 2025

Outline

- 1 Introduction & motivation
- 2 Project goals & methodology
- 3 Literature review
- 4 Data pipeline
- 5 Results & Analysis
- 6 Discussion & Future Work
- 7 Conclusion

Reliability engineering challenges

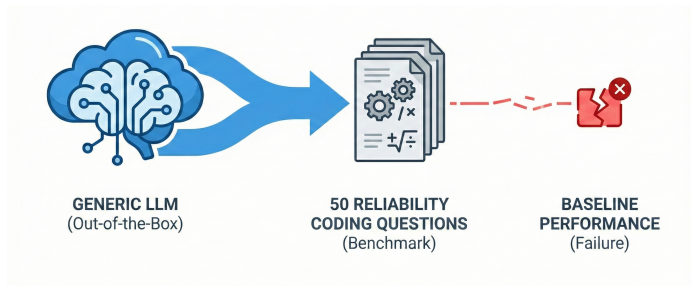
- Complex systems: nuclear plants, aircraft, electrical grids
- Requires advanced mathematical modeling
- Requires specialized domain knowledge

General LLMs fall short

- Built for general programming/tasks
- Poor performance on reliability-specific tasks
- Lack of specialized training data
- Cannot reason through multi-step domain problems

Key issue

State-of-the-art LLMs lack domain-specific data and are unable to reason through complex, multi-step reliability engineering problems effectively



The evaluation baseline:

- Previous students developed a benchmark of **50 hand-written reliability coding questions**
- State-of-the-art LLMs were evaluated "out-of-the-box" on this specific dataset

Project objectives

Primary goal

Develop a reliability-specific LLM specialized in coding tasks via self-instruct fine-tuning

Three key components:

① **Synthetic data generation**

Use self-instruct strategy to automatically generate large corpus of Q&A pairs

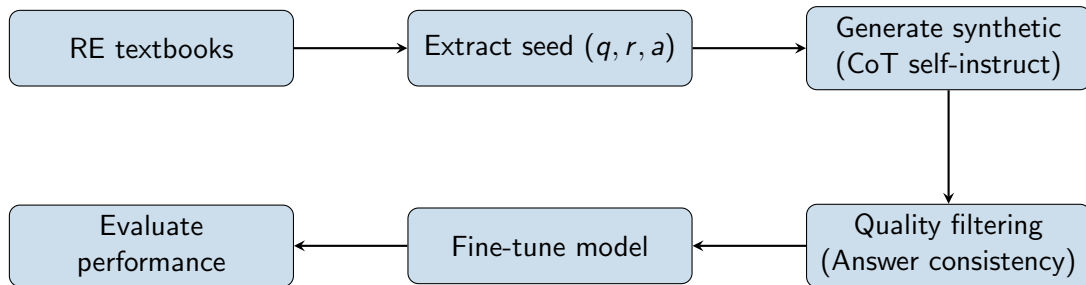
② **Chain-of-Thought integration**

Generate step-by-step reasoning for complex reliability problems

③ **Model fine-tuning & Evaluation**

Fine-tune open weight models to have domain knowledge and evaluate performance

Methodology overview



Self-instruct & Synthetic data generation

Self-Instruct (Wang et al., 2023)

Pioneering framework for bootstrapping instruction-following capabilities from LLMs themselves. Uses seed demonstrations to generate diverse synthetic instruction-output pairs, achieving 33% improvement over vanilla GPT-3 on Super-NaturalInstructions.

CoT-Self-Instruct (Yu et al., 2024)

Extends self-instruct by incorporating Chain-of-Thought reasoning during generation. Uses Answer-Consistency filtering for verifiable tasks. Outperforms original 'seed' datasets (s1k, OpenMathReasoning) on MATH500, AMC23, AIME24, and GPQA-Diamond benchmarks.

Relevance to our work: We adopt the self-instruct paradigm with consistency-based quality filtering to generate synthetic reliability engineering questions from textbook seeds.

Chain-of-Thought reasoning

Auto-CoT (Zhang et al., 2022)

Eliminates manual effort in creating CoT demonstrations by using diversity-based question clustering and automatic reasoning chain generation.

Key innovation:

- Cluster questions by similarity
- Generate reasoning chains with "Let's think step by step"
- Mitigate errors through diversity

Self-Refine IT (Ranaldi & Freitas, 2024)

Two-stage alignment: (1) Transfer reasoning via instruction-tuning on LLM demonstrations, (2) Self-refinement through Direct Preference Optimization (DPO). Enables smaller models to learn step-wise reasoning without separate reward models.

Results:

- Outperforms standard instruction-tuning
- Strong in/out-domain generalization
- Preference optimization for reasoning

Implication: CoT and preference-based optimization could be interesting next steps.

Two complementary question types

Exercise-based questions

- Numerical reliability problems
- Multi-step mathematical derivations

Derive the Mean Time To Failure (MTTF) for:

- a 2/3 voting system
- a 3/5 voting system

Assume negligible voter failure probability.

Answer: 5/6, 47/60

Theory-based questions

- Definitions, assumptions, interpretations
- “Why / How” reasoning
- Reliability theory understanding

Which of the following is Bayes Theorem for 2 events A, B ?

- A) $P(A|B) = P(B|A)P(A)/P(B)$
- B) $P(A|B) = P(A \cap B)/P(B)$
- C) ...

Answer: A

Step 1: Question/Answer extraction

- Scan textbooks for keywords such as (Exercise/Problem + Solution) with regex
- Extract all surrounding text

Step 2: Quality validation with SLM

- Remove samples that have or refer to figure/table references
- Validate self-containment and whether it meets our question requirements
- Verify clear, unambiguous answers

Step 3: Reformatting data with SLM

- Separate text into 3 components:
- Question
- Reasoning
- Answer

Result

288 high-quality seed (q, r, a) samples for baseline evaluation and synthetic generation

Synthetic data generation

Self-instruct strategy:

- 1 Sample 2 seed questions randomly
- 2 Prompt LLM to create novel inspired question
- 3 Generate step-by-step solution with CoT
- 4 Extract new (q, r, a)

Quality control - answer consistency checking:

- Re-solve generated question n times (with somewhat high temperature)
- Calculate consistency score: matches / total attempts
- Accept only both the score $\geq n/2$ and the majority prediction is equal to initial answer
- Filters out ambiguous or incorrectly formulated questions

Generation Results

Generated **81 synthetic questions** that passed consistency filtering

Baseline evaluation results

Exercise-based Q&A

Model	Dataset	Total	Correct	Accuracy
GPT-4-Mini	Seed	288	122	42.4%
Qwen3-14B	Seed	288	100	34.7%
GPT-4-Mini	Synthetic	81	74	91.4%
R1-Distill-Qwen3-14B	Synthetic	81	77	95.1%

Conceptual Q&A (MCQs)

Model	Total	Correct	Accuracy
GPT-4.1-Mini	231	219	95.0%
Qwen3-14B	231	214	92.7%

→ Conceptual questions already achieve near-ceiling performance → we focus fine-tuning efforts on exercise-based questions, where significant performance gains are still possible

Baseline evaluation results

Model	Dataset	Correct	Total	Accuracy
GPT-4-Mini	Seed	122	288	42.4%
R1-Distill-Qwen3-14B	Seed	100	288	34.7%
GPT-4-Mini	Synthetic	74	81	91.4%
R1-Distill-Qwen3-14B	Synthetic	77	81	95.1%

Key observation

Synthetic questions have **much higher** accuracy which makes sense as a similar capacity model created these questions

Open questions

Does this matter for finetuning? Does a smaller LM even have the capacity to learn questions generated by larger LM?

Fine-tuning setup & infrastructure

- Fine-tuning experiments conducted on **La Ruche** (CentraleSupélec HPC cluster)
- Dedicated training session on large-scale model fine-tuning
- Guidance on:
 - GPU job scheduling and resource allocation
 - Experiment monitoring and debugging

Training provided by **Solal** (CentraleSupélec)

Architecture: LoRA

- **Resource constraint:** Full fine-tuning of a 14B model requires massive GPU memory
- **Data constraint:** With a small dataset (~288 examples), retraining all weights risks **overfitting**

Solution: Freezing model weights and training only adapter layers

Implementation: Unsloth (optimization library)

- **Speed:** Accelerates training by **2x to 5x** thanks to optimized GPU kernels (manual backprop rewriting)
- **Memory efficiency:** Reduces VRAM usage by up to **50%**, preventing "Out-Of-Memory" errors

Impact: Allowed rapid iteration and debugging within the limited compute time

Fine-tuning results

Fine-tuning setup:

- Model: Qwen3-14B
- Training set: 230 seed Q&A — Testing set: 58 seed Q&A

Results for held out Q&A pairs

Model	Correct	Total	Accuracy	Change
Unfinetuned Qwen	5	58	8.6%	—
Finetuned Qwen	8	58	13.8%	+5.2%

Challenges encountered

① Data quality gap

- A good fine-tuning requires CoT and not only direct answers
- Models perform well on generated data but poorly on textbook questions
- Need better generation strategies to match complexity

② Limited training data

- Only 81 synthetic questions after filtering
- Original goal: thousands of examples
- Consistency filtering too strict? (50% threshold)

③ Domain complexity

- Reliability engineering requires deep statistical knowledge
- Numerical precision matters (small errors = incorrect)
- Requires human validation of generated Q&A

Future directions

Data generation

- Rework synthetic dataset approach
- Use textbook context as training material
- Improve consistency checking method (perhaps use a larger LM)
- Improve the generation chain-of-thought solutions of synthetic questions

Model training

- Full fine-tuning (not just LoRA)
- Multi-stage training pipeline
- Reinforcement learning from feedback

Evaluation

- Human expert evaluation on a sample of the seed/synthetic questions
- Code execution testing to see if the domain knowledge and problem solving skills generalize to parallel tasks
- Test real-world problem benchmarks

Key findings

- ❶ **Problem confirmed:** General LLMs perform poorly on reliability engineering tasks (8.6% baseline accuracy)
- ❷ **Fine-tuning works:** Domain-specific training improves performance (8.6% $\rightarrow$ 13.8%)
- ❸ **Data quality critical:** Synthetic questions much easier than real textbook (seed) problems (95% vs 14% accuracy)
- ❹ **Scale matters:** Current dataset too small for robust domain adaptation
- ❺ **Relevance of CoT:** Learning with reasoning steps is essential; otherwise, models learn to reproduce answers without learning the underlying problem-solving process.

Thank you for your attention!

Contact:

Alex Dalban & Elora Drouilhet
CentraleSupélec

Supervisor:

Dr. Zhiguo Zeng
`zhiguo.zeng@centralesupelec.fr`

References & Resources I

Key Papers:

- ① Wang, Y. et al. (2023). *Self-Instruct: Aligning Language Models with Self-Generated Instructions*. ACL 2023.
- ② Zhang, Z. et al. (2022). *Automatic Chain of Thought Prompting in Large Language Models*. ICLR 2023. arXiv:2210.03493.
- ③ Ding, B. et al. (2024). *Data Augmentation using Large Language Models: Data Perspectives, Learning Paradigms and Challenges*. ACL 2024 Findings. arXiv:2403.02990.
- ④ Ranaldi, L. & Freitas, A. (2024). *Self-Refine Instruction-Tuning for Aligning Reasoning in Language Models*. EMNLP 2024. arXiv:2405.00402.
- ⑤ Yu, P. et al. (2024). *CoT-Self-Instruct: Building High-Quality Synthetic Data for Reasoning and Non-Reasoning Tasks*. arXiv:2507.23751.
- ⑥ Liu, Z. et al. (2025). *Fin-R1: A Large Language Model for Financial Reasoning through Reinforcement Learning*. arXiv:2503.16252.

Models Used:

- Qwen3-14B (Alibaba Cloud)
- GPT-4-Mini (OpenAI)
- Mistral-Small-3.1-24B (Mistral AI)

Tools & Libraries:

- OpenRouter API
- Python (extractors, evaluators, generators)
- LoRA (Low-Rank Adaptation)